

OS Scheduling Simulator

Documentatie Proiect

Materia: Calitatea Sistemelor Software

Tehnologii: C#, .NET, Avalonia UI

Echipa:

- Filimon David-Christian
- Breaban Mihai
- Pintescu Sebastian-Dimitrie

25 mai 2026

Cuprins

1	Descriere generala	2
2	Tehnologii folosite	2
3	Structura proiectului	3
4	Descrierea modulelor principale	3
4.1	Activity	3
4.2	Process	3
4.3	Processor	4
4.4	MemoryManager	4
4.5	Scheduler	5
5	Algoritmul Round-Robin	5
6	Managementul memoriei	5
7	Testare unitara	6
8	Clase testate	6
9	Teste pentru Scheduler	6
10	Rezultatul testelor	7
11	Code Coverage	7
12	Probleme identificate prin testare	8
13	Faza 3 - Assertions	8
13.1	Assertions in Process.cs	8
13.2	Assertions in Processor.cs	8
13.3	Assertions in MemoryManager.cs	8
13.4	Assertions in Scheduler.cs	9
14	Cum se ruleaza proiectul	9
15	Cum se ruleaza testele	9
16	Cum se genereaza raportul de coverage	9
17	Contributia membrilor echipei	9
18	Concluzie	10

1. Descriere generala

Acest proiect reprezinta un simulator simplificat de sistem de operare, dezvoltat pentru materia **Calitatea Sistemelor Software**.

Aplicatia simuleaza executia unor procese utilizator pe unul sau mai multe procesoare, folosind un algoritm de planificare **Round-Robin preventiv**. Sistemul include si o simulare simplificata a memoriei virtuale, unde procesele pot fi incarcate in RAM sau mutate pe disc in functie de memoria disponibila.

Proiectul a fost dezvoltat in trei faze:

1. Faza 1 - dezvoltarea aplicatiei

- implementarea simulatorului;
- citirea configuratiei din fisier;
- executia proceselor pe procesoare;
- gestionarea memoriei;
- generarea logului de simulare;
- afisarea grafica printr-un grafic Gantt.

2. Faza 2 - unit testing

- testarea unitara a claselor;
- utilizarea mock-urilor;
- generarea raportului de code coverage.

3. Faza 3 - assertions

- integrarea assertions;
- verificarea preconditions, postconditions si invariants;
- utilizarea fault sniffers.

2. Tehnologii folosite

Proiectul este dezvoltat in **C#** folosind platforma **.NET**.

Pentru interfata grafica a fost utilizata biblioteca **Avalonia UI**.

Pentru testare au fost folosite:

- xUnit
- Moq
- coverlet.collector
- ReportGenerator

3. Structura proiectului

```

Calitatea Sistemelor Software
|
|-- OS.sln
|
|-- OS
|   |-- Core
|   |   |-- Activity.cs
|   |   |-- IMemoryManager.cs
|   |   |-- MemoryManager.cs
|   |   |-- Models.cs
|   |   |-- Process.cs
|   |   |-- Processor.cs
|   |   |-- Scheduler.cs
|   |
|   |-- ViewModels
|   |   |-- MainViewModel.cs
|   |
|   |-- Views
|   |   |-- MainWindow.axaml
|   |   |-- MainWindow.axaml.cs
|   |
|-- OS.Tests
|   |-- Core
|   |   |-- ActivityTests.cs
|   |   |-- MemoryManagerTests.cs
|   |   |-- ProcessTests.cs
|   |   |-- ProcessorTests.cs
|   |   |-- SchedulerTests.cs

```

4. Descrierea modulelor principale

4.1. Activity

Clasa `Activity` descrie o activitate a unui proces.

Tipuri de activitati:

```

Execution
SysCall

```

Fiecare activitate contine:

- Type
- Duration

4.2. Process

Clasa `Process` reprezinta un proces utilizator.

Contine:

- Id
- MemoryRequired
- ReleaseTime
- LastProcessorId
- CurrentStatus
- Activities

Starile procesului:

```
OnDisk  
Ready  
Running  
Blocked  
Finished
```

4.3. Processor

Clasa **Processor** reprezinta un procesor al sistemului.

Contine:

- Id
- CurrentProcess
- TimeSpentInSlice

4.4. MemoryManager

Clasa **MemoryManager** gestioneaza memoria RAM simulata.

Responsabilitati:

- calculeaza memoria libera;
- incarca procese in RAM;
- elimina procese cand memoria este insuficienta;
- calculeaza timpul de transfer.

Metoda principala:

```
EnsureInRam(Process p)
```

4.5. Scheduler

Clasa **Scheduler** reprezinta componenta principala a simulatorului.

Responsabilitati:

- gestioneaza timpul simularii;
- planifica procese;
- aplica Round-Robin;
- trateaza preemptia;
- genereaza evenimente.

Metoda principala:

```
Ticks()
```

5. Algoritmul Round-Robin

Round-Robin este algoritmul de planificare utilizat pentru procesele utilizator.

Fiecare proces primeste o cuanta de timp pe procesor. Daca procesul nu termina executia in acea cuanta, este preemptat si pus inapoi in coada ready.

Exemplu:

Cuanta = 5

Procesul P1 are nevoie de 12 unitati de timp.

Executia:

- P1 ruleaza 5 unitati
- P1 este preemptat
- P1 ruleaza inca 5 unitati
- P1 este preemptat
- P1 ruleaza ultimele 2 unitati

Evenimentul de preemptie este:

```
PREEMPTED
```

6. Managementul memoriei

Un proces poate fi executat doar daca este incarcat in RAM.

Daca memoria este insuficienta, **MemoryManager** elimina procese existente si calculeaza timpul de transfer.

Formula folosita:

```
memoryRequired * diskTransferRate
```

Implementarea actuala foloseste o politica apropiata de FIFO.

7. Testare unitara

Pentru faza 2 au fost create teste unitare intr-un proiect separat:

```
OS.Tests
```

Framework utilizat:

```
xUnit
```

Pentru mocking:

```
Moq
```

8. Clase testate

Clase testate:

- OS.Core.Activity
- OS.Core.Process
- OS.Core.Processor
- OS.Core.MemoryManager
- OS.Core.Scheduler
- OS.Core.SimulationEvent
- OS.ViewModels.MainWindowViewModel

Clase netestate:

- OS.App
- OS.Program
- OS.MainWindow

9. Teste pentru Scheduler

Pentru Scheduler, interactiunea cu MemoryManager a fost simulata folosind Moq si interfata IMemoryManager.

Testele verifica:

- planificarea proceselor;

- generarea evenimentelor;
- preemptia Round-Robin;
- transferul de memorie;
- resetarea schedulerului.

Exemplu de mocking:

```
var memoryMock = new Mock<IMemoryManager>();

memoryMock
    .Setup(m => m.EnsureInRam(It.IsAny<Process>()))
    .Returns(0);
```

10. Rezultatul testelor

Comanda folosita:

```
dotnet test
```

Rezultat:

```
Total tests: 64
Failed: 0
Succeeded: 64
Skipped: 0
```

Toate testele au fost executate cu succes.

11. Code Coverage

Coverage-ul a fost generat folosind:

```
dotnet test --collect:"XPlat Code Coverage"
```

Raport HTML:

```
reportgenerator -reports:"OS.Tests\TestResults\**\coverage.cobertura.xml"
```

Rezultate:

- Line coverage: 71.2%
- Branch coverage: 69.2%

Clase principale:

- Activity - 100%
- MemoryManager - 100%
- Process - 100%
- Processor - 100%
- Scheduler - 94.5%

12. Probleme identificate prin testare

Testele au evidentiat:

1. Activity permite durata negativa;
2. Process permite memorie negativa;
3. SimulationEvent permite valori invalide;
4. MemoryManager permite procese mai mari decat RAM-ul;
5. implementarea memoriei este apropiata de FIFO.

13. Faza 3 - Assertions

In faza 3 au fost inserate assertions direct in codul aplicatiei.

Assertions sunt verificari booleene executate la runtime.

Exemplu:

```
Debug.Assert(p != null,  
    "Procesul nu poate fi null");
```

Tipuri utilizate:

- Preconditions
- Postconditions
- Invariants
- Fault sniffers

13.1. Assertions in Process.cs

- verificarea parametrilor constructorului;
- validarea starii initiale;
- verificarea identicatorului procesului.

13.2. Assertions in Processor.cs

- verificarea id-ului procesorului;
- verificarea starii procesorului;
- invariant pentru procesor liber/ocupat.

13.3. Assertions in MemoryManager.cs

- validarea memoriei totale;
- verificarea transferului;
- invariant pentru memoria utilizata.

13.4. Assertions in Scheduler.cs

- validarea parametrilor;
- verificarea starii proceselor;
- verificarea transferului de memorie;
- verificarea resetarii schedulerului.

14. Cum se ruleaza proiectul

Build:

```
dotnet build
```

Rulare aplicatie:

```
dotnet run --project OS
```

15. Cum se ruleaza testele

```
dotnet test
```

16. Cum se genereaza raportul de coverage

Pasul 1:

```
dotnet test --collect:"XPlat Code Coverage"
```

Pasul 2:

```
reportgenerator -reports:"OS.Tests\TestResults\**\coverage.cobertura.xml"
```

Pasul 3:

```
start coverage\report\index.html
```

17. Contributia membrilor echipei

Proiectul a fost realizat prin colaborarea tuturor membrilor echipei, fiecare avand responsabilitatea principala asupra unei faze a proiectului si contributii suplimentare in celelalte etape de dezvoltare.

- **Filimon David-Christian** – coordonarea fazei 1, privind dezvoltarea simulatorului si implementarea functionalitatilor principale ale aplicatiei; contributii la fazele de testare si validare ale proiectului;
- **Breaban Mihai** – contributii principale in faza 2, prin dezvoltarea si extinderea testelor unitare, precum si implicare in implementarea si verificarea functionalitatilor aplicatiei;

- **Pintescu Sebastian-Dimitrie** – coordonarea fazei 3, prin integrarea mecanismelor de verificare folosind assertions (`preconditions`, `postconditions`, `invariants`, `fault sniffers`); contributii la dezvoltarea si testarea aplicatiei.

Pe parcursul proiectului, toate etapele au fost realizate colaborativ, membrii echipei participand la implementare, testare, depanare si revizuirea codului pentru asigurarea unei aplicatii stabile si coerente.

18. Concluzie

Proiectul demonstreaza implementarea unui simulator simplificat de sistem de operare, impreuna cu aplicarea conceptelor de:

- testare unitara;
- mocking;
- assertions;
- code coverage.

Aplicatia ofera o baza stabila pentru analiza algoritmilor de planificare si management al memoriei.